Implementación comparativa de la base de datos de la Red Social Estudiantil Pascualina

Tarea 4 — Informe técnico preliminar para revisión del equipo

1. Introducción

Este documento organiza la propuesta física de la Red Social Estudiantil Pascualina en PostgreSQL, MySQL y Microsoft SQL Server. La base de diseño es el informe de la Tarea 3, que describe diez tablas, trece claves foráneas, claves primarias simples y compuestas, restricciones de dominio y acciones referenciales. La implementación incorpora además el atributo semiestructurado datos_extra en perfil, de acuerdo con el documento complementario sobre JSONB.

2. Justificación de los SGBD seleccionados

PostgreSQL

Se conserva porque ya existe una propuesta del equipo y ofrece JSONB para almacenar documentos semiestructurados en formato binario, operadores de consulta e índices GIN. Es de código abierto y permite definir claves, CHECK y acciones referenciales. Como consideración, requiere conocer particularidades de sus tipos y operadores.

MySQL

Se propone MySQL 8.0.16 o posterior por su amplia adopción en aplicaciones web, disponibilidad de herramientas como MySQL Workbench y soporte del tipo JSON. La sintaxis de identidad utiliza AUTO_INCREMENT. Se debe comprobar la versión, ya que el soporte efectivo de CHECK depende de la versión.

Microsoft SQL Server

Se propone SQL Server 2016 o posterior, con SQL Server Management Studio como herramienta de administración. Para mantener compatibilidad amplia, datos_extra se representa mediante NVARCHAR(MAX) y se valida con ISJSON, en vez de asumir un tipo JSON nativo disponible en todas las versiones. La identidad se define con IDENTITY(1,1).

La selección es una propuesta comparativa para la tarea; la elección final debe confirmarse con el equipo y el docente. Los tres scripts deben ejecutarse en sus respectivos motores antes de afirmar que están verificados.

3. Criterios para mantener la continuidad con la Tarea 3

Se mantienen las diez tablas: estudiante, perfil, publicacion, grupo, evento, interes, estudiante_grupo, estudiante_evento, estudiante_interes y seguidor. También se mantienen las trece claves foráneas, la relación uno a uno entre estudiante y perfil, las relaciones muchos a muchos y la restricción que impide que un estudiante se siga a sí mismo.

Para resolver diferencias entre el script/Excel y el PDF de la Tarea 3, se priorizaron los tamaños y reglas explícitos del PDF: nombre y apellido VARCHAR(50), correo VARCHAR(100), programa VARCHAR(80), semestre SMALLINT, biografia VARCHAR(300), foto VARCHAR(150), contacto VARCHAR(100), contenido VARCHAR(500), nombre_grupo VARCHAR(80) sin UNIQUE, descripcion de grupo/evento VARCHAR(300), nombre_interes VARCHAR(50) UNIQUE y descripcion de interés VARCHAR(200). Esta reconciliación debe ser aprobada por el grupo porque cambia detalles presentes en el script y el Excel posteriores.

4. Implementación física y diferencias

PostgreSQL utiliza GENERATED BY DEFAULT AS IDENTITY, TIMESTAMP, JSONB y un índice GIN para datos_extra. MySQL utiliza AUTO_INCREMENT, DATETIME para fechas con hora, JSON y el motor InnoDB. SQL Server utiliza IDENTITY(1,1), DATETIME2 y NVARCHAR(MAX) con CHECK(ISJSON(datos_extra)=1). Las diferencias son adaptaciones físicas; las entidades y reglas de negocio se conservan.

Las acciones ON DELETE CASCADE se aplican a perfiles, publicaciones y tablas asociativas. RESTRICT en PostgreSQL y MySQL, y NO ACTION en SQL Server, protege los grupos y eventos cuando el creador u organizador aún está referenciado. ON UPDATE CASCADE se declara para mantener equivalencia con el diseño lógico, aunque en la práctica las claves identity suelen ser estables y no se actualizan en operaciones ordinarias. En SQL Server, las dos claves foráneas de seguidor usan NO ACTION para evitar múltiples rutas de cascada hacia la misma tabla; por ello, antes de eliminar un estudiante deben

eliminarse explícitamente sus relaciones de seguimiento dentro de una transacción. Esta es una diferencia física documentada frente al comportamiento en cascada de PostgreSQL y MySQL.

5. Uso de datos semiestructurados en Big Data e IoT

El campo perfil.datos_extra permite almacenar atributos opcionales y variables como habilidades, deportes e información de ubicación. Un documento de ejemplo podría contener arreglos de habilidades y deportes y un objeto ubicacion con latitud y longitud. Esto evita añadir una columna relacional cada vez que se incorpora una característica opcional del perfil.

En un caso de uso relacionado con IoT, una aplicación universitaria podría recibir ubicaciones o datos contextuales desde dispositivos móviles y guardar atributos variables como documento JSON. Para análisis geoespacial intensivo, validación de coordenadas, privacidad y grandes volúmenes, conviene evaluar tipos geoespaciales especializados, políticas de retención y una arquitectura de ingestión analítica; JSON por sí solo no resuelve esos problemas.

Las implementaciones no son idénticas: PostgreSQL usa JSONB, MySQL tiene JSON nativo y SQL Server almacena JSON como texto Unicode validado con ISJSON en las versiones consideradas. El índice GIN incluido es específico de PostgreSQL. En MySQL y SQL Server los índices sobre propiedades JSON deben diseñarse según las consultas y funciones disponibles en la versión instalada.

6. Pruebas y evidencias

Se incluye un archivo de pruebas para PostgreSQL con inserciones de ejemplo, consultas de JSONB y verificaciones de estructura. Las pruebas negativas de CHECK, UNIQUE y seguimiento propio están comentadas para que se ejecuten individualmente y no detengan el resto del archivo. El equipo debe ejecutar los scripts, guardar capturas de los resultados y registrar errores reales de cada motor.

Estado de validación: los archivos se han preparado y revisado estáticamente, pero no se afirma que MySQL ni SQL Server hayan sido ejecutados en sus respectivos motores. Tampoco deben afirmarse resultados exitosos en PostgreSQL hasta ejecutar el script de pruebas en la base de datos de trabajo.

7. Dificultades, ventajas y desventajas observadas

Dificultades técnicas previstas que deben contrastarse con la experiencia real: adaptar la sintaxis de identidad; elegir equivalentes para fechas; representar JSON de manera distinta en cada motor; y comprobar las acciones referenciales y restricciones en las versiones instaladas.

Ventajas comparativas esperadas: PostgreSQL ofrece JSONB y GIN; MySQL integra un tipo JSON y es común en aplicaciones web; SQL Server se integra con el ecosistema Microsoft y SSMS. Limitaciones a considerar: las características JSON y los índices varían entre motores; las herramientas, licencias y administración tienen costos de aprendizaje distintos; y las diferencias de sintaxis hacen necesario mantener scripts separados.

Esta sección debe completarse después de las pruebas reales con las dificultades efectivamente encontradas, capturas de errores y resultados. No se deben presentar las dificultades previstas como experiencias ya ocurridas.

8. Conclusiones y propuestas de mejora

Conclusión grupal — propuesta

La comparación muestra que un modelo relacional común puede implementarse en motores distintos, pero los tipos y algunas funciones deben adaptarse. Mantener un modelo lógico compartido y documentar las diferencias en los diccionarios reduce inconsistencias. La extensión JSON permite representar atributos variables, aunque debe complementarse con validación, índices adecuados y criterios de privacidad.

Conclusiones individuales — completar por cada integrante

Cada integrante debe escribir una reflexión personal en primera persona: qué implementó, qué dificultad concreta tuvo, qué aprendió sobre el DDL y qué decisión tomaría de otra manera. No debe copiarse una conclusión grupal como si fuera individual.

Propuestas de mejora

1) automatizar la creación de datos de prueba para los tres motores; 2) comparar tiempos de consulta con conjuntos de datos controlados; 3) definir índices según consultas reales; 4) documentar versiones exactas y capturas; 5) revisar seguridad y privacidad de datos de contacto y ubicación; 6) considerar tipos geoespaciales si la ubicación se convierte en una función central.

9. Organización del repositorio

La carpeta Tarea4 contiene diccionarios/diccionarios_tarea4.xlsx, ddl/postgresql.sql, ddl/mysql.sql, ddl/sql_server.sql, ddl/pruebas_postgresql.sql, este informe y una carpeta video pendiente del enlace de YouTube. Si la guía exige un archivo separado por SGBD, se deben exportar las hojas físicas del Excel a archivos independientes y mantener también el diccionario genérico actualizado.

Referencias

Elmasri, R. y Navathe, S. (2007). Fundamentos de sistemas de bases de datos (5.ª ed., pp. 317–344). Pearson Education.

Fadhel, S. A. y Jameel, E. A. (2022). A comparison between NoSQL and RDBMS: storage and retrieval. Minar International Journal of Applied Sciences and Technology, 4(3), 172–184. https://doi.org/10.47832/2717-8234.12.18

Hasan, M., Panidi, E. y Badenko, V. (2019). Comparative evaluation of NoSQL and relational databases performance while analyzing semi-structured geospatial data. GEOBALCANICA. https://doi.org/10.18509/gbp.2019.64

Microsoft Learn. (2025, 12 de junio). ¿Qué es SQL Server Management Studio (SSMS)? https://learn.microsoft.com/es-es/ssms/sql-server-management-studio-ssms

Treviño-Villalobos, M., Víquez-Acuña, L., Quirós-Oviedo, R. y Víquez-Acuña, O. (2022). Evaluación del tiempo de respuesta de un geoservicio utilizando una base de datos híbrida y distribuida. Revista Colombiana de Computación, 23(1), 34–43. https://doi.org/10.29375/25392115.4228